

Stallo e Attesa Indefinita

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi



- Problemi della concorrenza: deadlock e starvation
- Grafo di assegnazione delle risorse
- Metodi di gestione delle situazioni di stallo
- Metodi per prevenire lo stallo
- Metodi per individuare lo stallo e ripristino

Deadlock e Starvation

- **Deadlock** (Stallo) – due o più processi sono in attesa per un tempo indefinito di un evento che può essere causato solo da un processo a sua volta in attesa.

- Esempio: Siano S e Q due semafori inizializzati ad 1

P0	P1
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>
<code>⋮</code>	<code>⋮</code>
<code>signal(S);</code>	<code>signal(Q);</code>
<code>signal(Q)</code>	<code>signal(S);</code>

- **Starvation** (Attesa indefinita). Un processo potrebbe non essere mai rimosso dalla coda di attesa in cui si trova.



Stallo e Attesa Indefinita

STALLO

Deadlock (Stallo)

Il deadlock si verifica quando sono verificate tutte e quattro le seguenti condizioni.

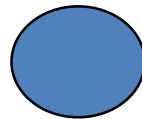
- **Mutua esclusione:** soltanto un processo alla volta può utilizzare una risorsa.
- **Possesso e attesa:** un processo in possesso di almeno una risorsa è in attesa di acquisire ulteriori risorse possedute da altri processi.
- **Senza prelazione:** una risorsa può essere rilasciata dal processo che la possiede solo volontariamente.
- **Attesa circolare:** esiste un insieme $\{P_0, P_1, \dots, P_0\}$ di processi in attesa tale che P_0 è in attesa di una risorsa posseduta da P_1 , P_1 è in attesa di una risorsa posseduta da P_2 , ..., P_{n-1} è in attesa di una risorsa posseduta da P_n , e P_0 è in attesa di una risorsa posseduta da P_0 .

Grafo di Assegnazione delle Risorse

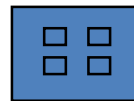
Un insieme di vertici V ed un insieme di archi E .

- V è partizionato in due sottoinsiemi:
 - $P = \{P_1, P_2, \dots, P_n\}$, l'insieme di tutti i processi del sistema.
 - $R = \{R_1, R_2, \dots, R_m\}$, l'insieme di tutti i tipi di risorse del sistema.
- Arco di richiesta – arco orientato $P_i \rightarrow R_j$
- Arco di assegnazione – arco orientato $R_j \rightarrow P_i$

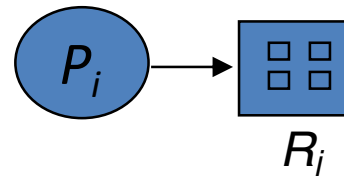
- Processo



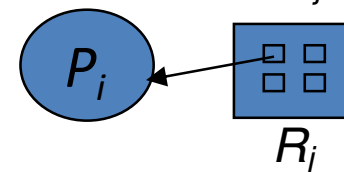
- Tipo di risorsa con 4 istanze



- P_i richiede una istanza di R_j

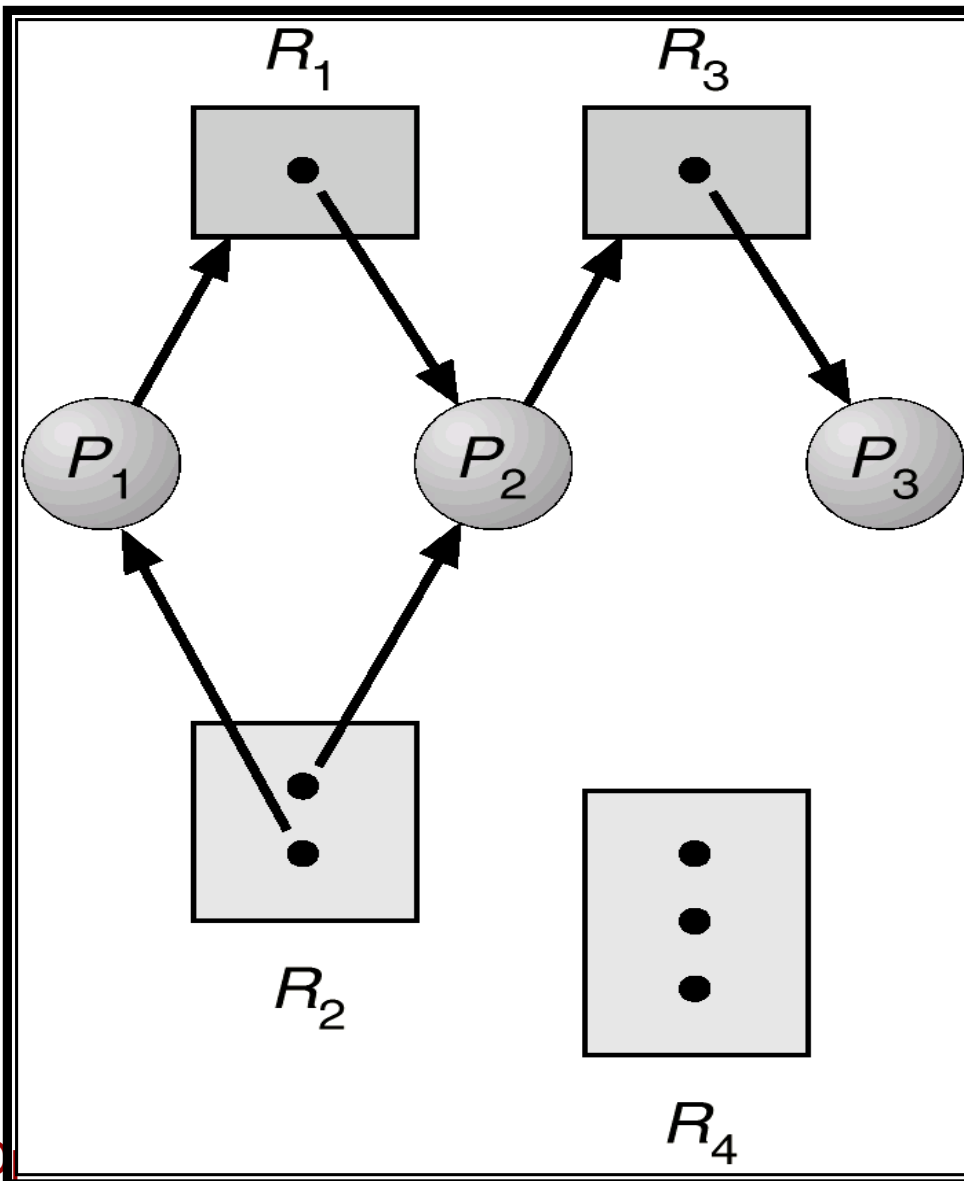


- P_i possiede una istanza di R_j

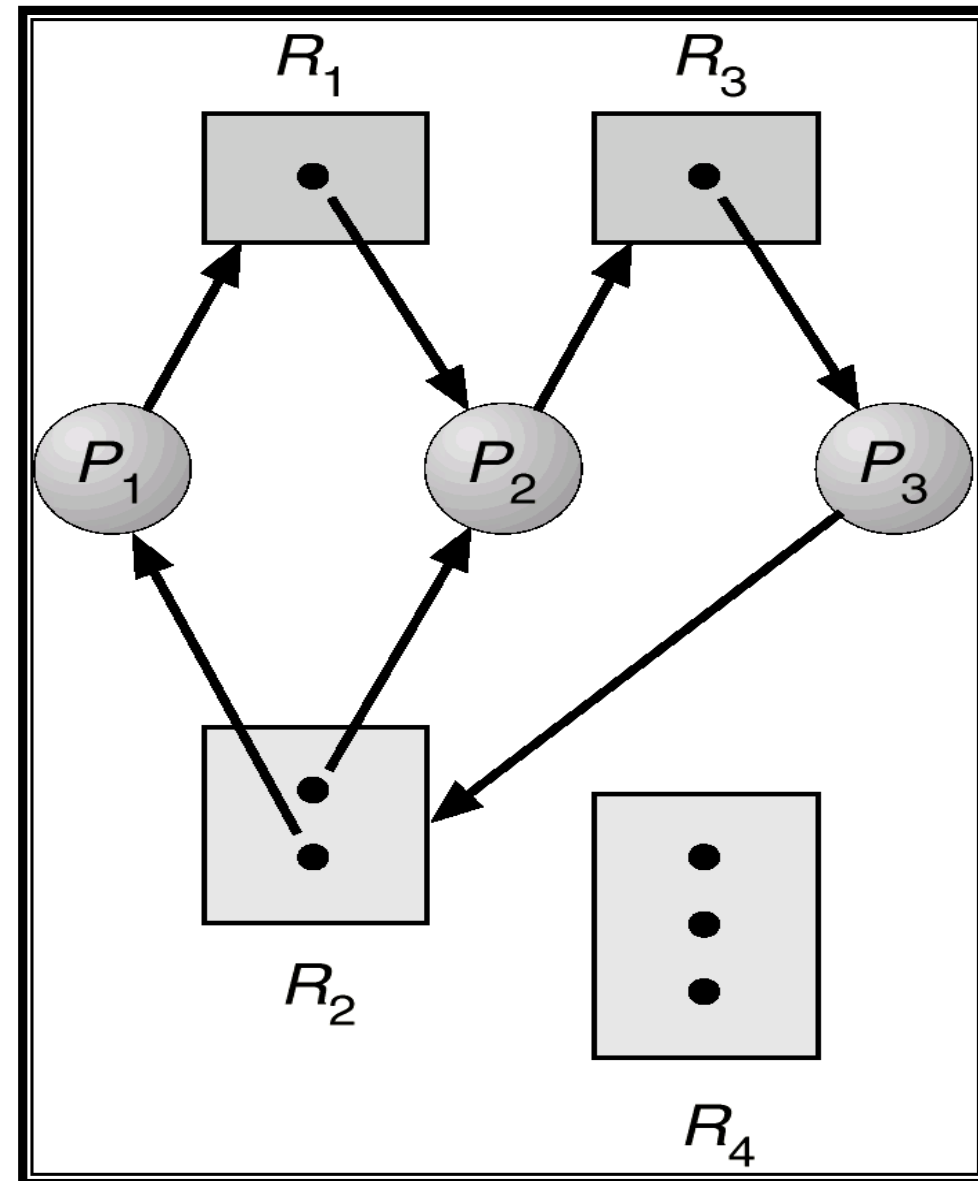


Grafo di Assegnazione delle Risorse

Senza stallo

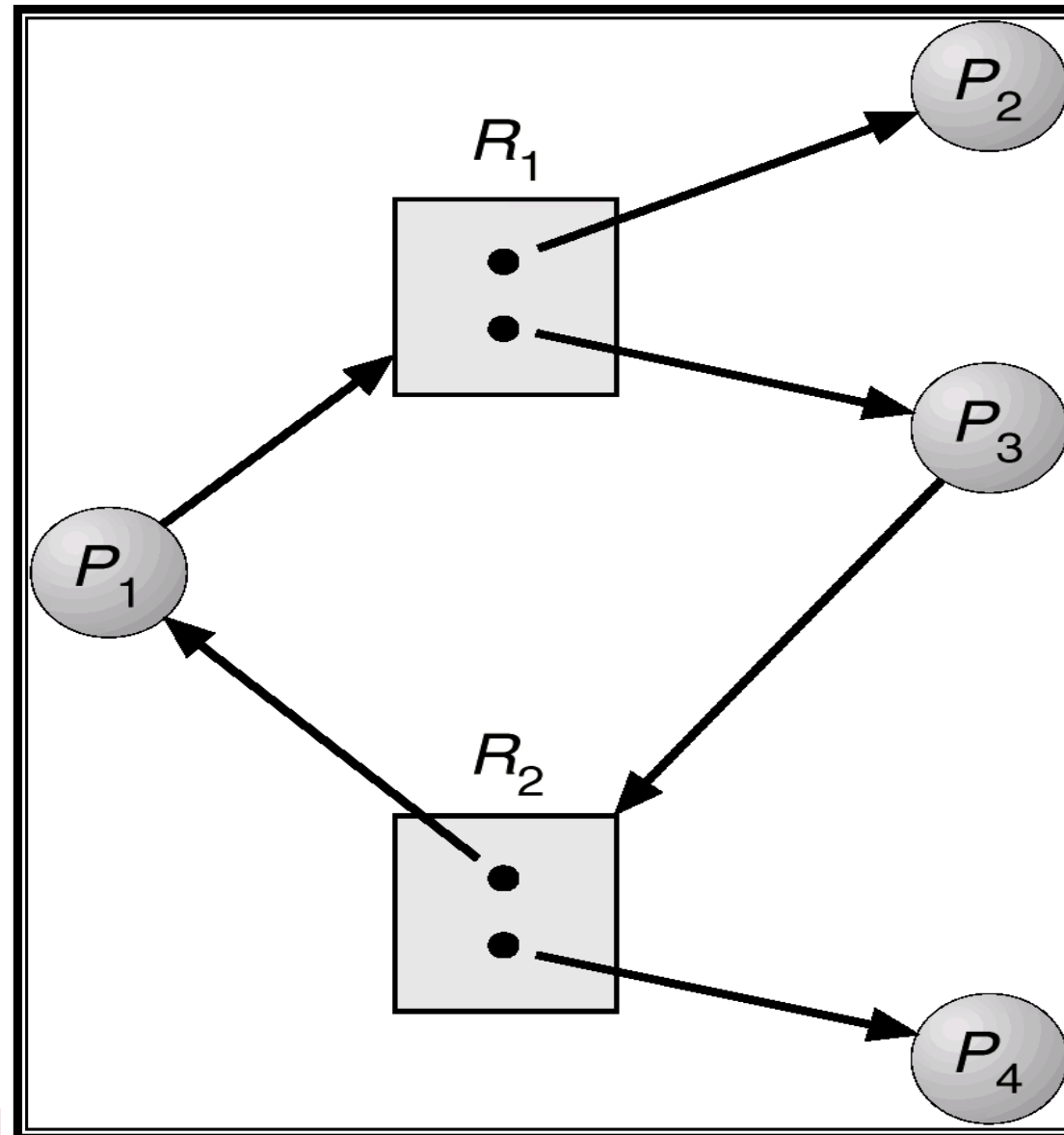


Con stallo



Grafo di Assegnazione delle Risorse

Grafo ciclico ma senza stallo



Grafo di Assegnazione delle Risorse

- Se il grafo non contiene cicli $\Rightarrow$ no deadlock.
- Se il grafo contiene un ciclo $\Rightarrow$
 - Se esiste una sola istanza per ogni tipo di risorsa coinvolta nel ciclo, allora si ha deadlock.
 - Se esistono più istanze per tipo di risorsa, potrebbe esserci la possibilità di deadlock (condizione necessaria ma non sufficiente).



Metodi di Gestione dei Deadlock

- Assicurarsi che il sistema non entrerà mai in situazione di stallo (servono metodi di prevenzione).
- Permettere lo stallo, ma se si verifica riconoscerlo e ripristinare una situazione corretta (servono metodi di riconoscimento e di ripristino).
- Ignorare il problema, partendo dal presupposto che lo stallo si verificherà molto raramente (usato da gran parte dei SO, Unix compreso).

Prevenzione del Deadlock

Negare almeno una delle quattro condizioni

- **Mutua Esclusione** – non richiesta per le risorse condivisibili; deve invece continuare a valere per le risorse non condivisibili.
- **Possesso e Attesa** – un processo quando richiede delle risorse non deve possederne già altre.
 - Un processo può richiedere tutte le risorse che gli servono prima di iniziare la sua esecuzione oppure può richiedere una risorsa quando non ne possiede nessuna.
 - Basso utilizzo delle risorse; possibilità di starvation.
- **Senza Prelazione** –
 - Se un processo che possiede delle risorse, richiede una nuova risorsa che non può essere ancora assegnata, gli vengono prelevate tutte le risorse possedute. Le risorse sono aggiunte alla lista delle risorse che il processo sta attendendo.
 - Il processo verrà riattivato solo quando tutte le risorse nella lista (quelle vecchie e quella nuova) sono disponibili.
 - Adatto a risorse il cui stato si può salvare e ripristinare facilmente.
- **Attesa Circolare** – imporre un ordinamento totale all'insieme di tutti i tipi di risorse e che un processo richieda le risorse seguendo l'ordine crescente.

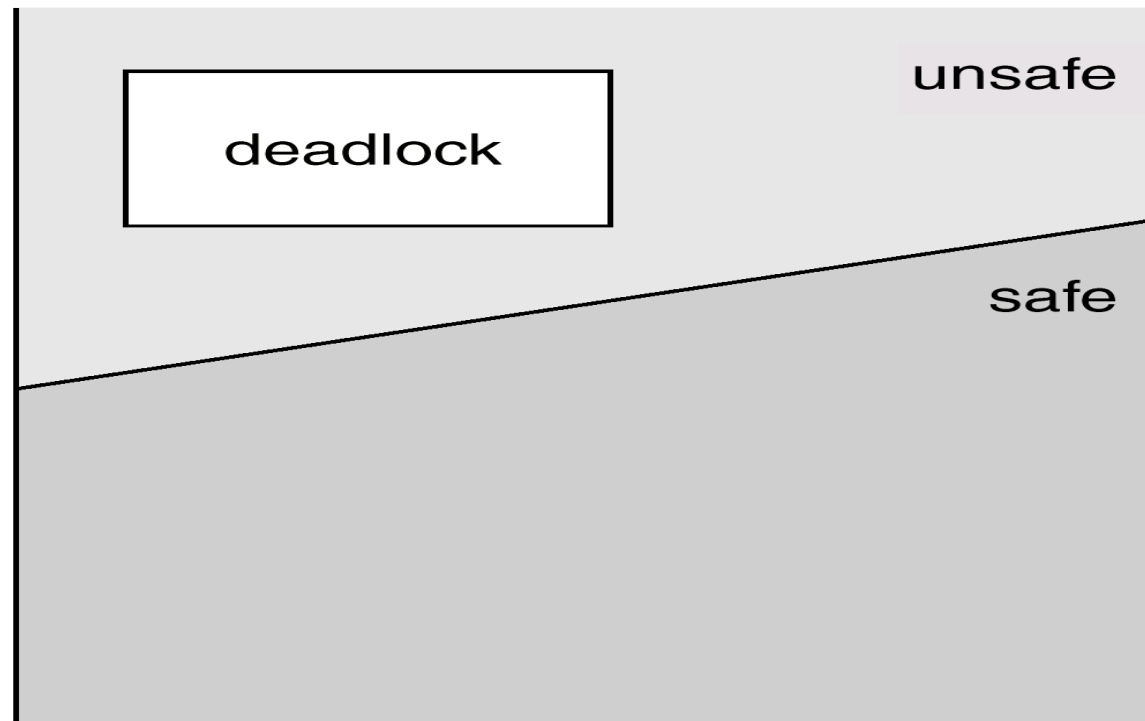
Algoritmi per evitare lo stallo

- I metodi descritti precedentemente hanno tutti effetti collaterali negativi. Un metodo alternativo consiste nel richiedere ulteriori informazioni ai processi quando richiedono le risorse.
 1. Ogni processo deve dichiarare il massimo numero di risorse di ogni tipo di cui può avere bisogno.
 2. L' algoritmo per evitare lo stallo esamina dinamicamente lo stato di assegnazione delle risorse e garantisce che non si entri mai in condizione di attesa circolare.
 3. Lo stato di assegnazione delle risorse è definito dal numero di risorse disponibili, dal numero di risorse assegnate e dalle richieste dei processi.

- Uno stato si dice sicuro se impedisce il verificarsi di uno stallo.
- Il sistema è in uno stato sicuro se esiste una sequenza di processi sicura.
- Una sequenza $\langle P_1, P_2, \dots, P_n \rangle$ è sicura se per ogni P_i , le risorse che P_i può ancora richiedere possono essere soddisfatte dalle risorse attualmente disponibili + le risorse possedute da tutti i P_j , con $j < i$.
 - Se le risorse di cui P_i ha bisogno non sono immediatamente disponibili, allora P_i può attendere fino a quando tutti i P_j hanno terminato.
 - Quando tutti i P_j hanno terminato, P_i può ottenere tutte le risorse di cui ha bisogno, completare il compito assegnato e terminare.
 - Quando P_i termina, P_{i+1} può ottenere le risorse di cui ha bisogno. E così via.

Stati sicuri, non sicuri e di stallo

- Stato sicuro $\Rightarrow$ niente stallo.
- Stato non sicuro $\Rightarrow$ possibilità di stallo.



- Gli algoritmi per evitare lo stallo in realtà evitano che il sistema finisca in uno stato non sicuro.

Algoritmo con grafo di allocazione delle risorse

-Risorsa a istanza singola

Variante del grafo di allocazione delle risorse

- Arco di reclamo $P_i \rightarrow R_j$ indica che il processo P_j può richiederla risorsa R_j in un qualsiasi momento futuro; è rappresentato da una linea tratteggiata.
- Un arco di reclamo diventa un arco di richiesta quando il processo richiede la risorsa.
- Quando una risorsa è rilasciata, l'arco di assegnazione diventa ridiventa un arco di reclamo.

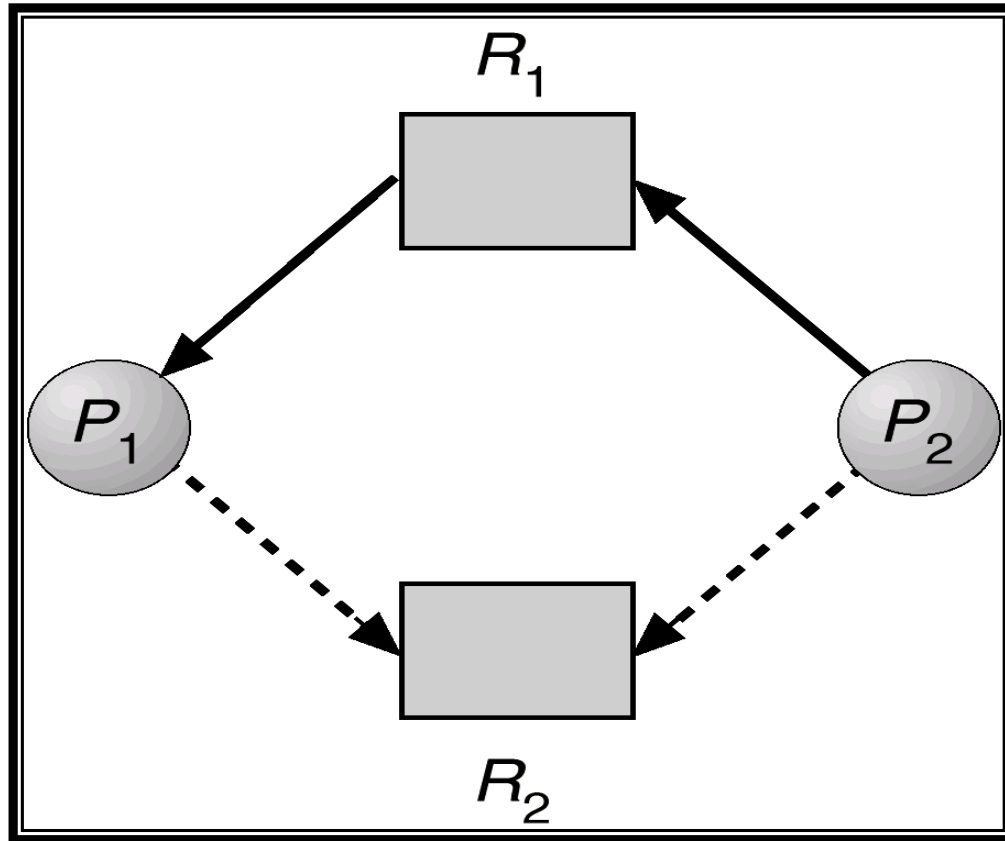
Algoritmo

- Le risorse devono essere reclamationate a priori nel sistema.
- Una richiesta si può soddisfare se e solo se la conversione dell'arco di richiesta $P_i \rightarrow R_j$ in arco di assegnazione $R_j \rightarrow P_i$ non causa la formazione di un ciclo.
- La ricerca di cicli nel grafo ha complessità $O(n^2)$.

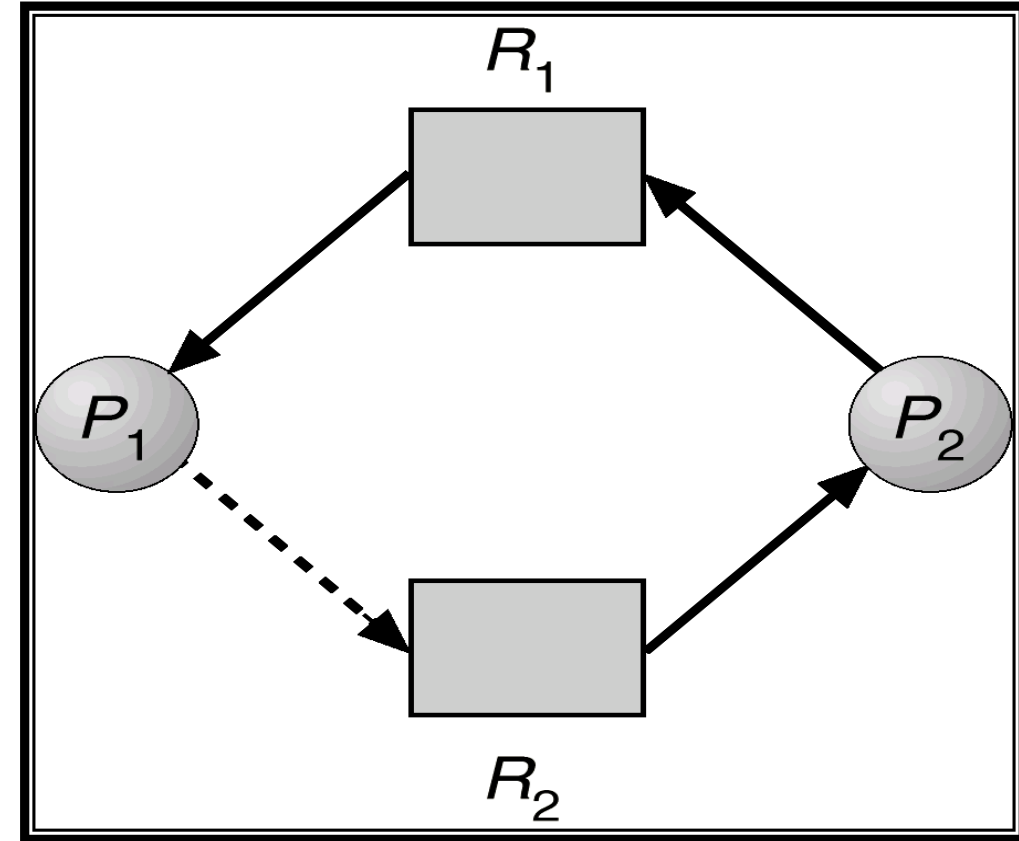
Algoritmo con grafo di allocazione delle risorse

-Risorsa a istanza singola

Stato sicuro



Stato non sicuro

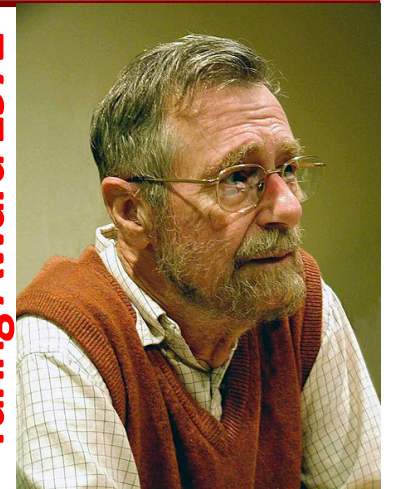


Algoritmo del banchiere

Risorse a istanza multipla

- Quando si presenta, un nuovo processo deve dichiarare il numero massimo di istanze di cui necessita per ogni tipo di risorsa.
- Quando un processo richiede una risorsa bisogna controllare se lascia il sistema in uno stato sicuro, altrimenti deve attendere.
- Quando un processo ottiene le risorse le deve rilasciare in un tempo finito.

Turing Award 1972



E.W. Dijkstra

- n processi – m tipi di risorse
- Strutture dati
 - Available: vettore di lunghezza m . Se $\text{Available}[j] = k$, ci sono k istanze ancora disponibili delle risorse di tipo R_j .
 - Max: matrice $n \times m$. Se $\text{Max}[i,j] = k$, allora il processo P_i può richiedere al massimo k istanze della risorsa R_j .
 - Allocation: matrice $n \times m$. Se $\text{Allocation}[i,j] = k$ allora a P_i sono attualmente allocate k istanze di R_j .
 - $\text{Allocation}[i]$ è l' i -esima riga della matrice
 - Need: matrice $n \times m$. Se $\text{Need}[i,j] = k$, allora P_i può aver bisogno ancora di altre k istanze di R_j per completare il suo compito.
 - $\text{Need}[i]$ è l' i -esima riga della matrice
- Tra vettori vale il seguente ordinamento: $v \leq w$ se e solo se $\forall i . v[i] \leq w[i]$
- $\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$.

Check State

- Work vettore di interi di lunghezza m;
- Finish vettore di Booleani di lunghezza n.
- 1. Initialize:
 - Work = Available
 - Finish [i] = false for $i = 1, 3, \dots, n$.
- 2. Find an index i such that both:
 - Finish [i] = false
 - $\text{Need}[i] \leq \text{Work}$
- 3. If no such i exists, go to step 4.
 - Work = Work + Allocation[i]
 - Finish[i] = true
 - go to step 2.
- 4. If Finish [i] == true for all i,
 - then the system is in a safe state
 - else the system is in an unsafe state

Check Request

- Request[i] = request vector for process P_i .
 - If Requesti [j] = k then process P_i wants k instances of resource type R_j .
- 1. If Request[i] $\leq$ Need[i] then go to step 2.
else raise error condition
 - since process has exceeded its maximum claim.
- 2. If Request[i] $\leq$ Available then go to step 3.
else P_i must wait
 - since resources are not available.
- 3. Simulate to allocate requested resources to P_i :
 - Available = Available - Requesti;
 - Allocationi = Allocationi + Requesti;
 - Needi = Needi – Requesti;
- 4. **Check State**
 - If safe $\Rightarrow$ the resources are allocated to P_i .
 - If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Algoritmo del banchiere

Un esempio

- 5 processi: P0 - P4;
3 tipi di risorsa: A (10 istanze), B (5 istanze), C (7 istanze).

Situazione all'istante t_0 :

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	[3 3 2]		
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

Situazione all'istante t_0 :

- $\text{Need} = \text{Max} - \text{Allocation}$

Need			
	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

- Il sistema è in uno stato sicuro perché la sequenza $\langle P1, P3, P4, P2, P0 \rangle$ soddisfa il criterio di sicurezza.

Situazione all'istante t_1 :

- Supponiamo ora che arrivi la richiesta di P1: Request1 = (1,0,2)
- $(1,0,2) \leq (1,2,2)$ richiesta compatibile con i bisogni (Need[1])
- $(1,0,2) \leq (3,3,2)$ richiesta compatibile con le disponibilità (Available)
- Quindi si ottiene il seguente nuovo stato:

	Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	4	3	2	3	0
P1	3	0	2	0	2	0			
P2	3	0	2	6	0	0			
P3	2	1	1	0	1	1			
P4	0	0	2	4	3	1			

- L'algoritmo di verifica mostra che
<P1, P3, P4, P0, P2> soddisfa i requisiti di stato sicuro.



Esercizio 4: Algoritmo del banchiere

- Partendo dallo stato finale dell' esempio precedente verificare se:
 - può essere soddisfatta la richiesta di P4
Request4 = (3,3,0)
 - può essere soddisfatta la richiesta di P0
Request0 = (0,2,0)



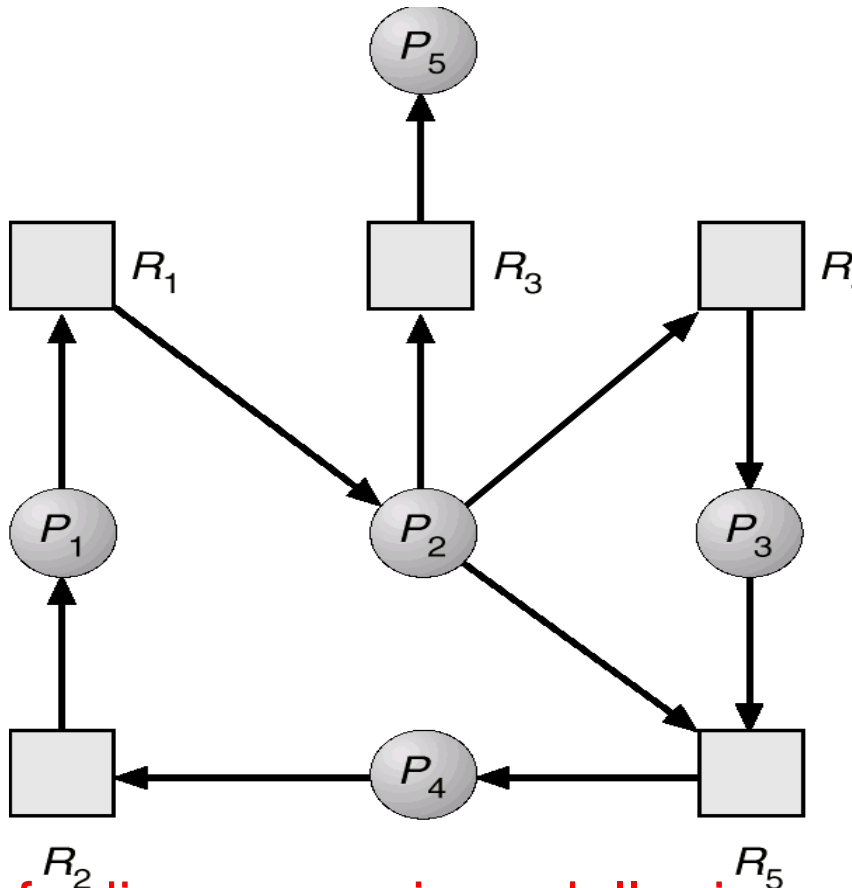
Il sistema può entrare in stallo

- Algoritmo di rilevamento
- Schema di ripristino

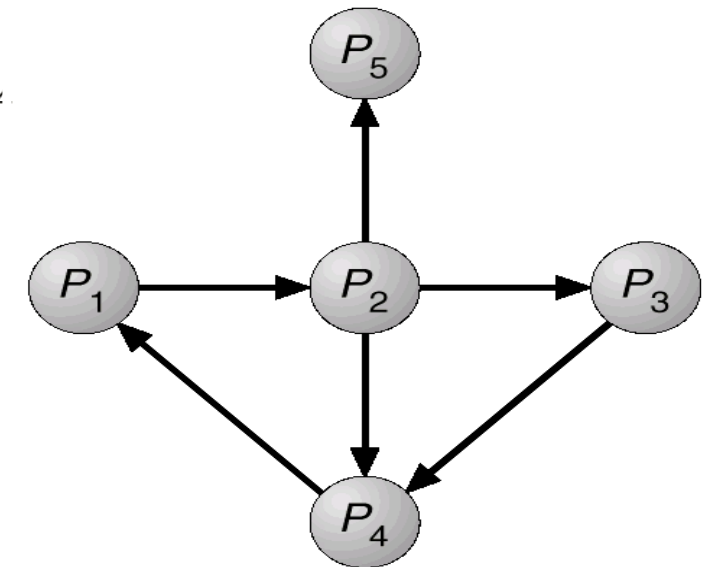
Tipi di Risorsa a Istanza Singola

- Usa i grafi di attesa.
 - i nodi sono processi
 - $P_i \rightarrow P_j$ se P_i attende una risorsa posseduta da P_j .
- Periodicamente si ricercano cicli nel grafo.

La complessità dell' algoritmo è $O(n^2)$.



Grafo di assegnazione delle risorse



Grafo di attesa

Tipi di Risorsa a Istanza Multipla

- n processi – m tipi di risorse

Strutture dati

- Available: vettore di lunghezza m che indica il numero di istanze disponibili per ciascun tipo di risorsa.
- Allocation: Matrice $n \times m$ che definisce il numero di istanze per ogni tipo di risorsa correntemente assegnate a ciascun processo.
 - Allocation[i] è l' i -esima riga della matrice
- Request: Matrice $n \times m$ che indica la richiesta corrente di ogni processo.
matrix indicates the current request of each process.
Request [i, j] = k significa che il processo P_i sta richiedendo altre k istanze della risorsa di tipo R_j .
 - Requesti è l' i -esima riga della matrice
- Tra vettori vale il seguente ordinamento
 - $v < w$ iff $\forall i . v[i] < w[i]$

Algoritmo di Rilevamento

- Work vettore di interi di lunghezza m ;
- Finish vettore di Booleani di lunghezza n .

1. Initialize:

- Work = Available
- For $i = 1, 2, \dots, n$,
if Allocation[i] $\neq 0$, then Finish[i] = false; else Finish[i] = true..

2. Find an index i such that both:

- Finish [i] = false
- Request[i] $\leq$ Work

3. If no such i exists, go to step 4.

- Work = Work + Allocation[i]
- Finish[i] = true
- go to step 2.

4. If Finish [i] == true for all i ,

- then the system is deadlock free
- else the system is deadlocked

La complessità dell' algoritmo è $O(m \times n^2)$.

I processi in stallo sono tutti i P_i tali che Finish[i]==false

Algoritmo di Rilevamento Esempio / 1

- 5 processi: P0 - P4;
3 tipi di risorsa: A (7 istanze), B (2 istanze), e C (6 istanze).

Situazione all'istante t_0 :

	Allocation			Request			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	0	0	0	0	0	0
P1	2	0	0	2	0	2			
P2	3	0	3	0	0	0			
P3	2	1	1	1	0	0			
P4	0	0	2	0	0	2			

- Eseguendo l'algoritmo per la sequenza <P0, P2, P3, P1, P4> risulta $\text{Finish}[i] = \text{true}$ per ogni i .

- Ipotizziamo che P2 chieda una risorsa C e che P1 chieda una risorsa C in meno

Situazione all'istante t_1 :

	Allocation	Request	Available
	A B C	A B C	A B C
P0	$\begin{pmatrix} 0 & 1 & 0 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 0 \end{pmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$
P1	$\begin{pmatrix} 2 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & 0 & \mathbf{1} \end{pmatrix}$	
P2	$\begin{pmatrix} 3 & 0 & 3 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & \mathbf{1} \end{pmatrix}$	
P3	$\begin{pmatrix} 2 & 1 & 1 \end{pmatrix}$	$\begin{pmatrix} 1 & 0 & 0 \end{pmatrix}$	
P4	$\begin{pmatrix} 0 & 0 & 2 \end{pmatrix}$	$\begin{pmatrix} 0 & 0 & 2 \end{pmatrix}$	

- Anche se si possono reclamare le risorse possedute dal processo P0, ciò non è sufficiente per soddisfare le richieste degli altri processi.
- Si verifica una situazione di stallo per i processi P1, P2, P3, e P4.

Algoritmo di Rilevamento: Utilizzo

- Quando e quanto spesso invocare l'algoritmo dipende da:
 - Quanto spesso si presume che uno stallo si verifichi,
 - Quanti processi sarebbero coinvolti dallo stallo
- Non è conveniente richiamare l'algoritmo arbitrariamente, poiché nel grafo di assegnazione delle risorse possono coesistere diversi cicli contemporaneamente, così da rendere difficile stabilire quali processi hanno causato lo stallo.



Ripristino: Terminazione dei Processi

- Terminare tutti i processi in stallo.
- Terminare un processo alla volta fino a quando lo stallo è rimosso (molto oneroso per il SO). L'ordine con cui scegliere i processi da terminare si può basare su:
 - Priorità dei processi.
 - Il tempo di computazione trascorso dall'inizio e il tempo ancora necessario per la terminazione del processo.
 - Quante e quali risorse il processo sta usando.
 - Di quante e quali risorse il processo ha ancora bisogno.
 - Il numero di processi che si devono terminare.
 - Il tipo di processo (interattivo o a lotti?).



Ripristino: Prelazione delle Risorse

- Selezione di una vittima – allo scopo di minimizzare i costi.
- Rollback (ripristino di un precedente stato sicuro) – il processo a cui viene sottratta una risorsa, deve essere riportato in uno stato sicuro da cui ripartire.
- Starvation (attesa indefinita) – bisogna evitare che un processo venga scelto sempre come vittima.



Stallo e Attesa Indefinita

ATTESA INDEFINITA



- Starvation o attesa indefinita è il nome assegnato al posticipo indefinito dell'assegnazione delle risorse richieste da un processo quando le risorse, sebbene disponibili per l'allocazione, non vengono mai assegnate a questo processo.

- Le decisioni sull'assegnazione delle risorse vengono prese localmente.
 - Non considerare i requisiti complessivi delle risorse del sistema può portare a delle anomalie.
- Le priorità dei processi.
- La selezione “casuale”.
- Non ci sono abbastanza risorse.

- Ci deve essere un manager indipendente per ogni risorsa che gestisca in modo “fair” la risorsa.
- Non dovrebbero essere applicate le priorità in modo rigido.
 - Aumentare la priorità di un processo al crescere del tempo di attesa (invecchiamento o **aging**).
 - Implementare le priorità assieme al razionamento:
 - considerare le priorità non come indicazioni di assoluta importanza, ma come misure della proporzione delle risorse che possono essere consumate.
- Evita selezioni casuali, concorrenza incontrollata, ecc.
- Fornire più risorse.



- Stabilire una soglia al tempo di attesa di ogni processo
- Quando il tempo di accesso di un processo supera la soglia si può ipotizzare che il processo sia in starvation
- Se i processi in attesa da molto tempo sono più di uno si può ipotizzare che sia in deadlock